

Reproducible documents with Quarto

Scientific workflows: Tools and Tips 

2023-05-11

What is this lecture series?

Scientific workflows: Tools and Tips

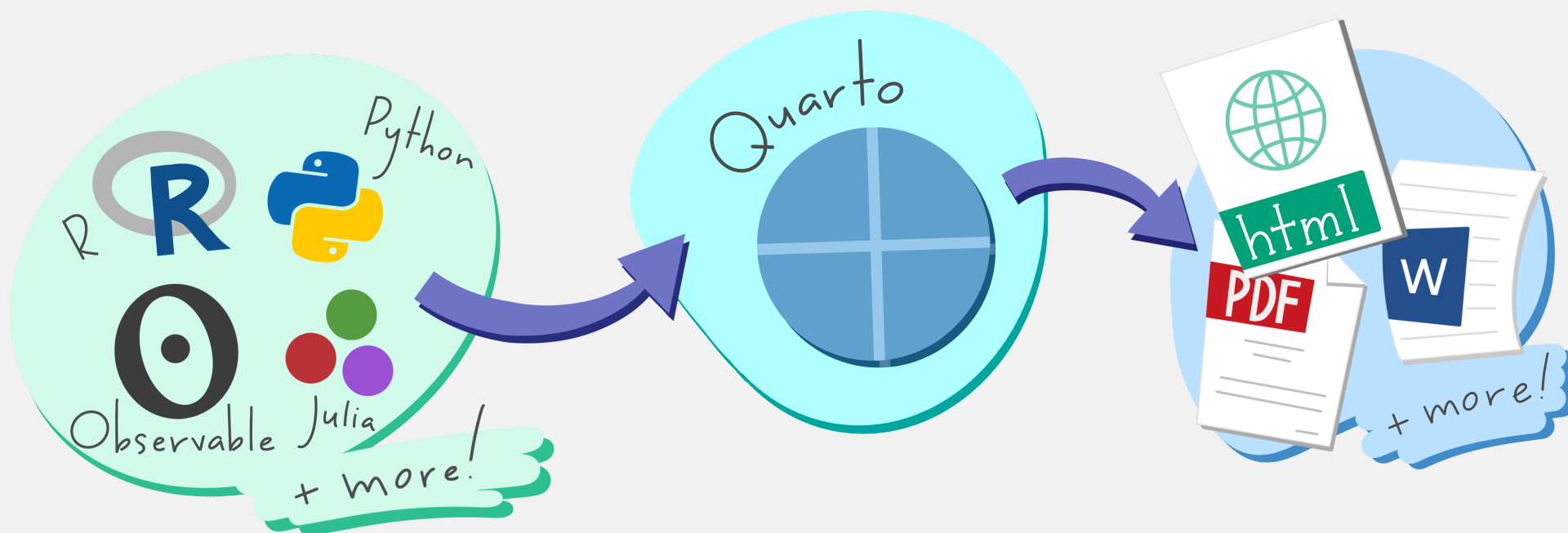
 Every 3rd Thursday  4-5 p.m.  Webex

- One topic from the world of scientific workflows
- For topic suggestions [send me an email](#)
- If you don't want to miss a lecture
 - Check out the [lecture website](#)
 - [Subscribe to the mailing list](#)
- Slides provided [on Github](#)

Reproducible documents with Quarto

Quarto is an open-source scientific and technical publishing system

Basic idea: Create documents with dynamic content and text



Artwork from "Hello, Quarto" keynote by Julia Lowndes and Mine Çetinkaya-Rundel, presented at RStudio Conference 2022. Illustrated by [Allison Horst](#)

Reproducible documents with Quarto

Document types that can be created with Quarto (examples):

- Documents: HTML, PDF, Word
- Presentations: HTML, Powerpoint
- Books: HTML, ePub, PDF
- Websites

Before we start

Quarto is a huge topic and there are so many cool Quarto things!

Goal of today: **Introduction** to Quarto and an **overview** of different document types and their possibilities.

- Focus on R and R Studio
- Keep in mind: This also works with other languages and other IDEs, the principles are all the same.

How to get Quarto

Different options, depending on your workflow:

- **Integrated** in new versions of **R Studio** (Update R Studio via [Help -> Check for Updates](#))
- Download the **CLI** for use with other IDE (e.g. Visual Studio code)

Check out the [Quarto website](#) for download and more info.

The classic use case

Reproducible documents for data analysis

An example document

An HTML example

Table of contents	The penguins of Antarctica	
1 Introduction	AUTHOR	PUBLISHED
2 Methods	Selina Baldauf	May 9, 2023
3 Results		
4 References		

1 Introduction

There are three main penguin species in Antarctica (*Chinstrap*, *Gentoo*, *Adelie*). You can see them in [Figure 1](#):

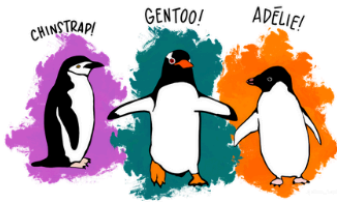


Figure 1: Illustration of the three penguin species by Allison Horst

In this paper we want to answer the following questions

1. How bill depth depends on bill length?
2. Which penguin species has the highest body mass?

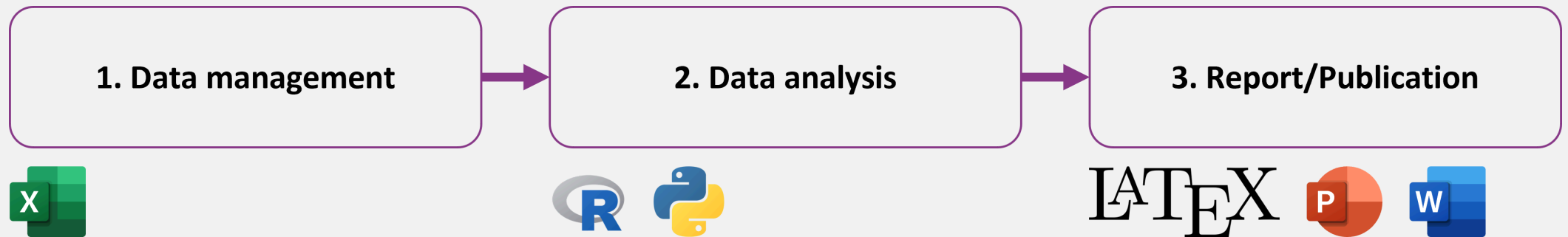
2 Methods

All analysis was done using R version 4.1.3 ([R Core Team 2022](#)) and the R markdown package ([Allaire et al. 2021](#)).

2.1 The data

The data was collected on islands in Antarctica and published by Gorman, Williams, and Fraser ([2014](#)). You can find the original paper with the title "Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus *Pygoscelis*)" ([Gorman Williams](#)

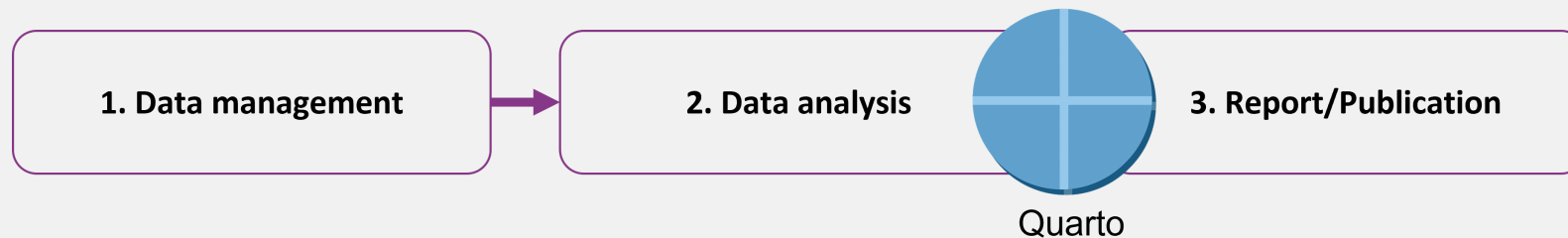
A standard workflow



Problem: Manual updates are **error prone** and **non-reproducible**

A Quarto workflow

Solution: Use a **Quarto workflow** → everything (code, text, metadata) in one place. Let **Quarto** do the magic



Advantages of this workflow:

- Easy to redo analysis
- No more copy pasting
- Reproducible

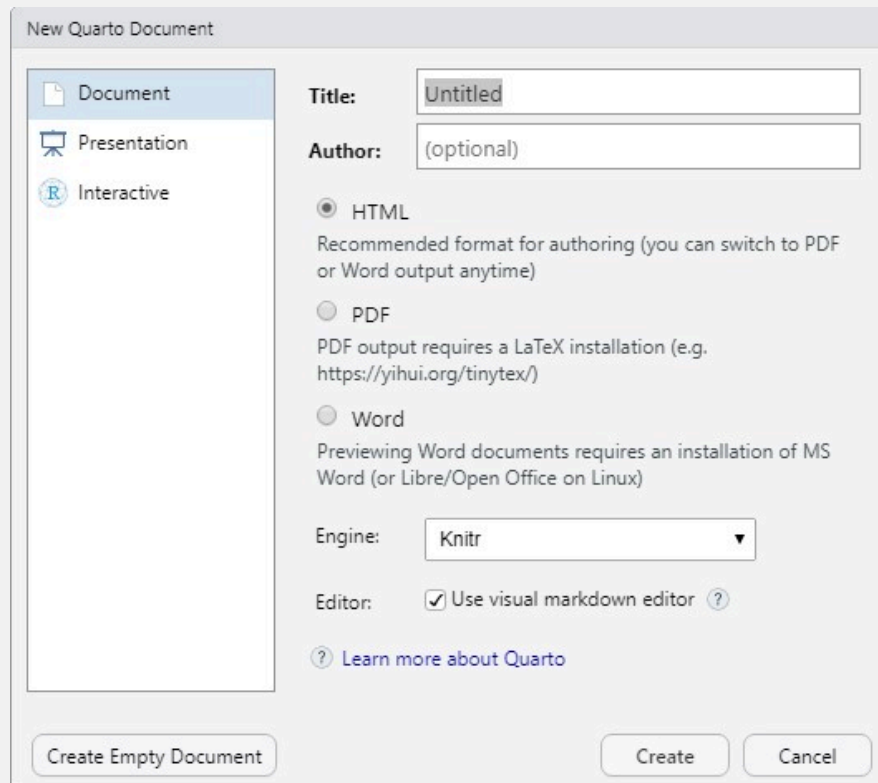
Download a quarto demo project from [Github](#)

The basic Quarto workflow

1. Create a `.qmd` document

Step 1 - Create the document

Open R Studio and go to **File -> New File -> Quarto Document**



Just click **Create** and the file will open in R Studio.

In other environments you can just create an empty file with **.qmd** ending

The basic Quarto workflow

1. **Create** a `.qmd` document
2. **Write** the document
 - *text* e.g. introduction, methods, or discussion
 - *code* (R, Python, Julia) that produces numbers, figures, tables, ...
 - *metadata* that defines how the document should look like (e.g. which output format)
3. **Render** the document to a defined output format (e.g. PDF) using `Quarto`

Step 3 - Render the document

- Click the **Render** button in R Studio
- Keyboard shortcut **Ctrl + Shift + K**
- Call the **quarto::quarto_render()** function
- In the terminal: **quarto render doc.qmd**

Basic Elements of a `.qmd` document

Text body, Code, YAML header

References for all the elements

- [Markdown syntax reference](#)
- Code chunks:
 - [R code](#)
 - [Python code](#)
- YAML header options:
 - [HTML](#)
 - [PDF](#)
 - [DOCX](#)

The text body - Markdown

Markdown is a simple markup language to create formatted text, you can e.g.

- Make italic *text* with `*text*` or bold **text** with `**text**`
- Generate headers of different levels

```
# Header level 1
## Header level 2
### Header level 3
```

- Create bullet lists

```
A bullet point list

- item 1
- item 2
- item 3
```

The text body - Markdown

You can also do more complex things like:

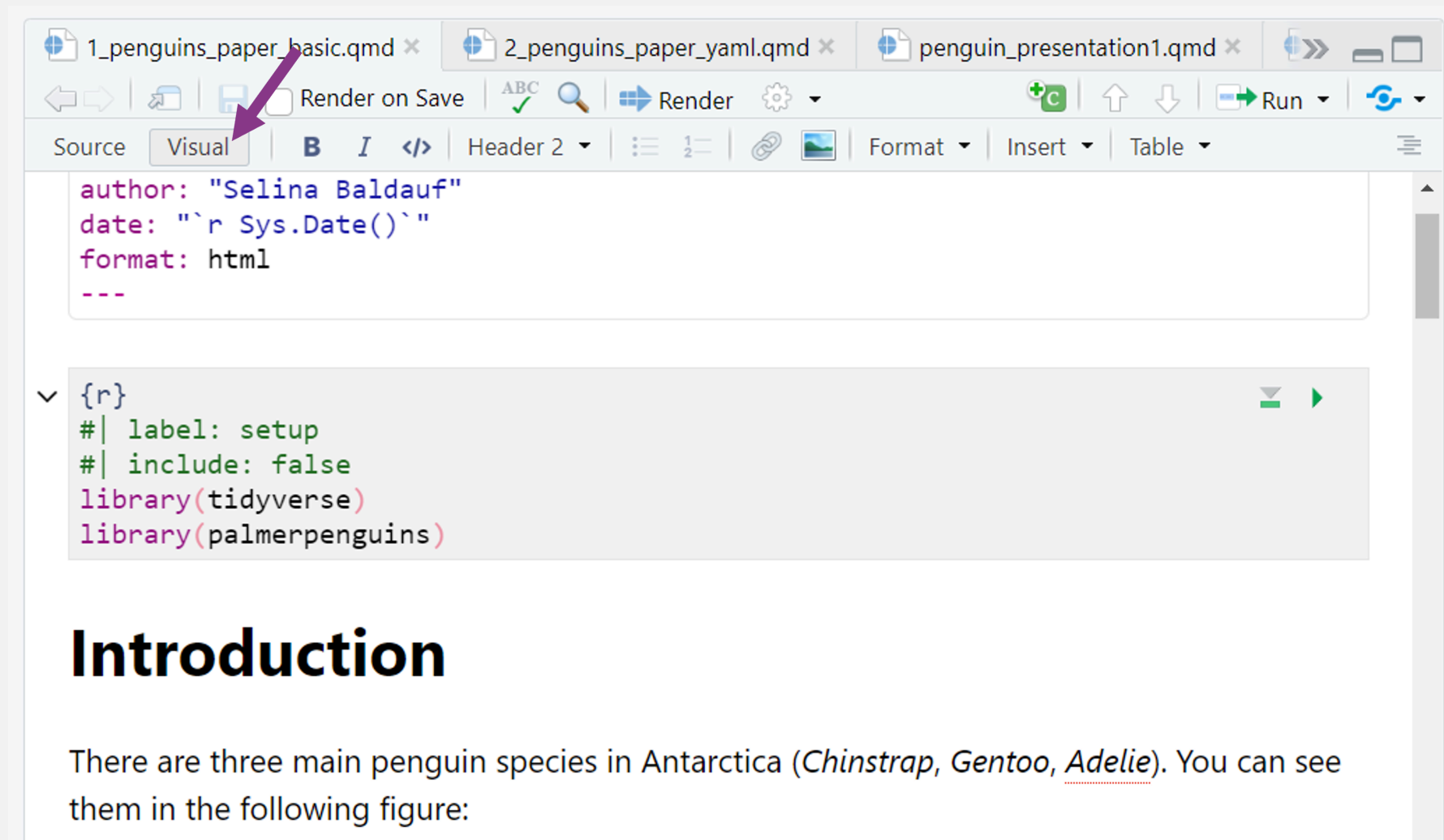
- Including images, links or footnotes
- Adding citations
- Latex style mathematical formulas

The text body - Markdown

If you don't want to use markdown, there is a really nice feature in R Studio: **The visual editor.**

The visual editor in R Studio

Convenient, word-like interface for formatting text and adding features.



The visual editor in R Studio

Using the visual editor, makes many things that would be painful in Markdown really easy.

My favorite feature in the visual editor:

- Add citations (**Insert -> Citation**) from Zotero library, DOI search, PubMed, ...

The Code

Code can be included in **code chunks** or as **inline code**

The Code

Inline code starts and ends with 1 backtick

```
`r`
```

Example

```
The mean of the values 1, 2 and 3 is `r mean(1:3)`
```

looks like this:

The mean of the values 1, 2 and 3 is 2.

The Code

Code chunks starts and ends with 3 backticks

```
```{r}
library(ggplot2)

ggplot(airquality, aes(Temp, Ozone)) +
 geom_point() +
 geom_smooth(method = "loess")
```
```

```
```{python}
import numpy as np
import matplotlib.pyplot as plt

r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r
fig, ax = plt.subplots(subplot_kw = {'projection': 'polar'})
ax.plot(theta, r)
ax.set_rticks([0.5, 1, 1.5, 2])
ax.grid(True)
plt.show()
```
```

The Code

Insert a code chunk (R Studio)

- Insert a code chunk by going to **Code -> Insert chunk**
- Use the keyboard shortcut **Ctrl + Alt + I / Cmd + Option + I**
- Inline chunks have to be typed or use the **</>** symbol in visual mode

The Code

Run code chunk

- Code chunks are run when document is rendered
- Code chunks can also be run like “normal” code
- Run Code chunk by clicking on the green arrow next to the chunk

```
```{r cars}
summary(cars)
```
```

A small icon for running a code chunk, consisting of a green downward-pointing triangle and a green rightward-pointing triangle.

The code

Code chunk have special comments that start with `#|` and that control the behaviour of the chunk.

```
```{r}
#| label: fig-airquality
#| fig-cap: Temperature and ozone level.
#| include: false

library(ggplot2)

ggplot(airquality, aes(Temp, Ozone)) +
 geom_point() +
 geom_smooth(method = "loess")
```
```

- **label**: Figure and chunk label that can be referred to in text
- **fig-cap**: Figure caption
- **include**: Include the output (i.e. the plot) in the document but don't show the code

YAML header

For Metadata

```
---  
title: "My first document"  
subtitle: "Whatever subtitle makes sense"  
author: "Selina Baldauf"  
date: "`r Sys.Date()`"  
---
```

- Inline R code can print the current date at render time

YAML header

For document output formats

```
---  
title: "My first document"  
author: "Selina Baldauf"  
date: "`r Sys.Date()`"  
format: html  
---
```

You can also specify multiple output formats

```
---  
title: "My first document"  
author: "Selina Baldauf"  
date: "`r Sys.Date()`"  
format:  
  html: default  
  pdf: default  
  docx: default  
---
```

YAML header

For document options

- Some options are shared between formats, some are specific to one format
- Be careful to get the indentation right!

```
---  
title: "My first document"  
author: "Selina Baldauf"  
date: "`r Sys.Date()`"  
format:  
  html:  
    number-sections: true  
    toc: true  
    toc-location: left  
  pdf:  
    toc: true  
    number-sections: true  
  docx: default  
---
```

YAML header

Execute options

- Default options for code chunks
- Can be overwritten by local comments in code chunks

```
---  
title: "My first document"  
author: "Selina Baldauf"  
date: "`r Sys.Date()`"  
format: html  
execute:  
  message: false  
  warning: false  
---
```

Summary

- Quarto combines formatted text and code to create reproducible documents

Use cases for scientists

- Data analysis reports and documents
- Use `qmd` instead of `.R` scripts to add text to the code (e.g. for interpretation or method description)
- Use it for teaching material
- ...

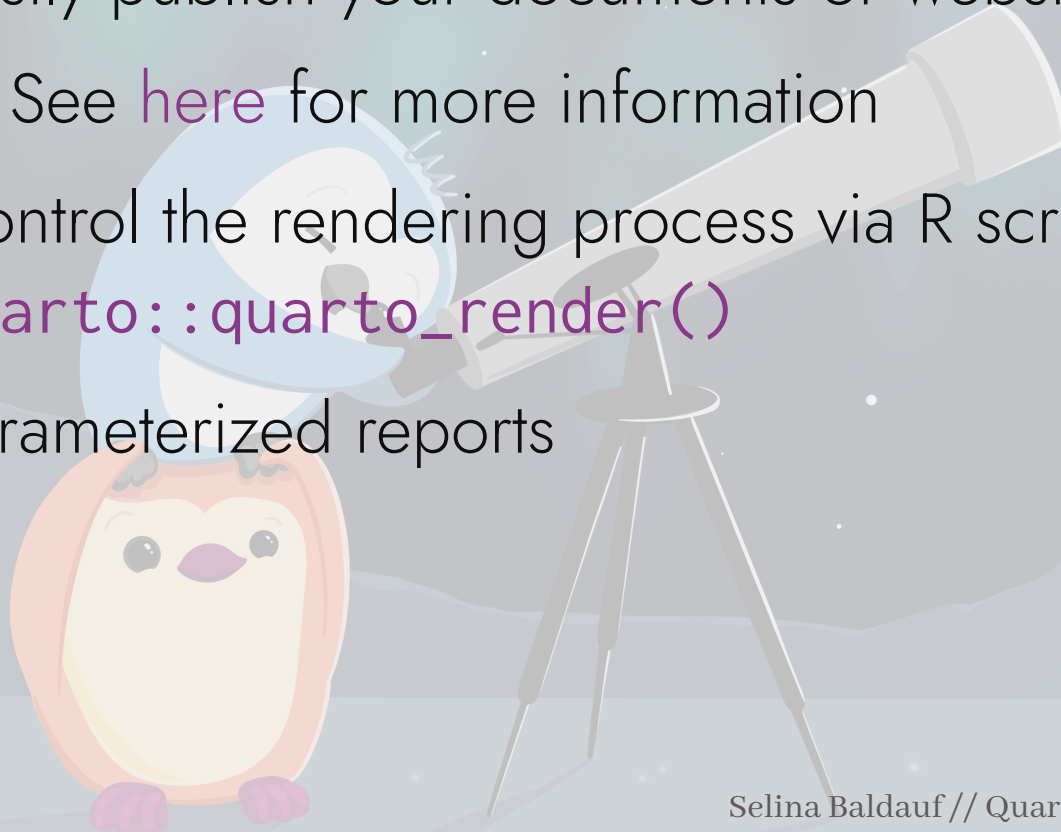
Outlook



Outlook

Quarto offers many more things like:

- Presentations (Powerpoint or Revealjs/HTML)
- Websites
- Easily publish your documents or websites on [quartopub](#) or Github
 - See [here](#) for more information
- Control the rendering process via R scripts using `quarto::quarto_render()`
- Parameterized reports



Next lecture

Version control with Git

Git is an essential skill if you use any programming language. It allows you to keep track of changes over time, collaborate with others, and maintain a clear and organized file structure. This can save time, improve research efficiency, and makes it easy to publish your code.

. . .

 15th June  4-5 p.m.  Webex

 [Subscribe to the mailing list](#)

 For topic suggestions and/or feedback [send me an email](#)

Thank you for your attention :)

Questions?



References

- [Quarto website](#) offers everything you need to get started
 - [Download Quarto and starting guide](#) for different IDEs
 - [Guides for different output formats](#)
 - [Gallery with Examples](#)
- [Quarto introduction workshop](#) on Youtube
- [A curated collection of resources](#)